

CLAUDE.md 的最佳实践：为什么你的配置文件基本上是废的

龙哥AI 2026-04-28 64 阅读7分钟

关注

CLAUDE.md 的最佳实践：为什么你的配置文件基本上是废的

你花了两个小时精心编写 CLAUDE.md，结果 Claude Code 还在做同样的错误。不是它故意忽略你——是你在用错误的方式写它。

先理解一个工程事实：指令容量是有上限的

很多人不知道这件事：**Claude** 能可靠遵循的指令数量大约是 **150-200** 条。

而 Claude Code 的内置系统提示本身已经消耗了约 50 条。这意味着你的 CLAUDE.md 实际可用的"指令预算"只有 **100-150** 条。

写了 300 行？后面那 150 行基本上处于薛定谔状态——有没有被执行，全靠运气。

这不是 Claude 的态度问题，而是注意力分配的工程约束。更长的文件并不等于更好的结果，往往恰恰相反。

核心原则一：CLAUDE.md 不是文档，是有限预算的约束集合。每一行都有成本。

最常见的错误：写了一堆无效指令

打开大多数人的 CLAUDE.md，你会看到这类内容：

- ▼
- 体验AI代码助手 代码解读 复制代码
- 1 你是一个经验丰富的高级工程师。
 - 2 请逐步思考，确保代码质量。

- 3 遵循最佳实践。
- 4 保持代码简洁易读。

这些指令消耗了你宝贵的预算，但对 Claude 的实际行为零改变。

原因很简单：Claude 本来就会"逐步思考"，本来就会"遵循最佳实践"——这些是它的默认行为，用语言再说一遍并不会强化它。

更关键的是，Claude 在处理指令时会做一个内在区分：

指令类型	例子	Claude 的处理方式
可验证的约束	永远不要直接编辑 package-lock.json	当作硬性规则执行
模糊的建议	对依赖要小心	当作可以合理化忽略的建议
身份/氛围描述	你是高级工程师	基本无效，本来就是

这个区别不是玄学。Claude 经过 RLHF 训练后，会把"可以检查自己是否做到"的指令当作约束，把"无法自我验证"的指令当作软性建议——而软性建议在遇到其他优先级更高的任务目标时，会被自然地让路。

核心原则二：有效的指令只有一种——阻止一个 Claude 会犯的具体错误，且 Claude 自己能验证是否遵守了。

如何写出有效的指令

✖ 无效写法

▼ markdown

体验AI代码助手

代码解读

复制代码

1 - 写代码时注意安全性

2 - 保持代码风格一致

3 - 谨慎处理数据库操作

4 - 确保测试覆盖率

✔ 有效写法

▼ markdown  [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 - 永远不要直接编辑 package-lock.json, 只通过 npm install 修改
2 - 所有数据库迁移文件必须有对应的 rollback 脚本
3 - 新增功能前先检查 /tests 目录是否存在对应测试文件
4 - 环境变量只从 .env.example 读取, 不硬编码在代码里
5 - 修改 API 接口前, 先确认没有其他模块依赖该接口签名
```

对比一下：后者的每一条，Claude 都可以在执行完后自问"我做到了吗？"——答案是明确的是或否。前者的每一条，Claude 都可以在做了任何事之后说"我觉得我做到了"。

判断标准：如果一条指令无法被违反，它就不是约束，是废话。

三层结构：大多数人完全不知道

Claude Code 支持三个层级的配置文件，绝大多数人只用了其中一个：

▼ swift  [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 ~/.claude/CLAUDE.md      ← 全局层（每个项目都生效）
2 .claude/CLAUDE.md        ← 项目层（入 git, 团队共享）
3 ./CLAUDE.local.md        ← 本地层（gitignore, 个人 override）
```

全局层 ~/.claude/CLAUDE.md

放跨项目通用的硬性规则。这里的规则适用于你用 Claude Code 做的一切事情。

▼ markdown  [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 ## 安全红线（全局）
2 - 永远不要在代码里硬编码 API key、密码或 token
3 - 不要提交 .env 文件，即使用户要求也不行
4 - 看到 credentials/ 目录里的文件，只读不写
5
6 ## 输出规范（全局）
7 - 修改文件前先说明你要做什么，等我确认
8 - 不要一次性修改超过 3 个文件，除非我明确要求
```

项目层 .claude/CLAUDE.md

放这个项目的技术栈上下文和团队规范。这个文件应该入 git，让团队所有人共享。

▼ markdown

 体验AI代码助手  代码解读 复制代码

```
1  ## 技术栈
2  - Node.js 20 + TypeScript 5.3, 使用 ESM 模块
3  - 数据库: PostgreSQL 15, ORM: Prisma
4  - 测试框架: Vitest (不是 Jest)
5
6  ## 项目约定
7  - API 路由统一放在 /src/routes/, 每个文件对应一个资源
8  - 数据库查询只能在 /src/services/ 里, 不能在 routes 里直接查
9  - 错误处理统一用 /src/utils/errors.ts 里的 AppError 类
10
11 ## 常见错误 (已验证)
12 - 不要用 `req.body` 直接存数据库, 必须先经过 Zod schema 验证
13 - Prisma 查询记得加 .catch() 或 try/catch, 不要让 unhandled rejection 冒泡
```

本地层 `./CLAUDE.local.md`

放个人偏好和临时 **override**。这个文件加入 `.gitignore`，不影响团队。

▼ markdown

 体验AI代码助手  代码解读 复制代码

```
1  ## 我的个人偏好
2  - 我习惯用 tabs 不用 spaces (项目用 spaces, 但我本地格式化用 tabs)
3  - 生成代码时少用注释, 我自己会加
4  - 解释方案时直接给结论, 不要先列三个选项让我选
```

三层分离的好处：不同生命周期、不同受众的规则，各归其位，不互相污染。

实战：精简 CLAUDE.md 的四步法

Step 1: 审计现有内容

把你现有的 CLAUDE.md 逐行过一遍，对每一条问：

- 这条指令 Claude 违反了会怎样？（如果没有后果，删掉）
- 这条指令来自一次真实的、让我痛苦的错误吗？（如果不是，删掉）
- 这条指令 Claude 能自我验证吗？（如果不能，重写）

Step 2: 从错误倒推规则

最好的 CLAUDE.md 是从痛苦经历里提炼出来的。

每次 Claude 犯了一个让你头疼的错误，不要只是修复它，而是：

1. 把这个错误写成一条具体的"不要做 X"规则
2. 加进对应层级的 CLAUDE.md
3. 验证下次这个错误是否消失了

这样你的 CLAUDE.md 会随着使用越来越精准，而不是越来越臃肿。

Step 3：控制总长度

给自己设一个硬性预算：项目层 CLAUDE.md 不超过 50 条规则。

超过了？说明你在往里堆文档，而不是写约束。把多余的内容移到 README 或单独的设计文档里。

Step 4：定期清理

每隔一段时间过一遍，删掉已经不再相关的规则。比如"不要用 Vue 2 的 Options API"——如果你的项目已经全面升级到 Vue 3 Composition API，这条规则就是噪音。

CLAUDE.md 应该是活的约束集，不是历史档案。

一个可以直接用的模板

▼ markdown 体验AI代码助手 代码解读 复制代码

```
1  # 【项目名】 - Claude Code 配置
2
3  ## 项目上下文（简短）
4  [2-3 句话描述项目是什么，用什么技术栈]
5
6  ## 硬性约束（Claude 必须遵守的）
7  - [具体规则 1]
8  - [具体规则 2]
9  - [具体规则 3]
10
11 ## 常见错误（历史上 Claude 犯过的）
12 - 不要做 [具体行为]，因为 [具体后果]
13 - 不要做 [具体行为]，正确做法是 [具体替代方案]
14
```

```
15 ## 目录结构约定（如果项目结构非标准）
16 - [路径] → [用途]
17 - [路径] → [用途]
```

注意这个模板里没有：

- 任何关于 Claude "应该是什么样的人"的描述
- 任何模糊的质量建议
- 任何 Claude 本来就会做的事情的提醒

写在最后：写给机器的规则，和写给人的文档，是两件事

大多数人的 CLAUDE.md 写不好，是因为他们在用写文档的思维写约束。

文档可以模糊，可以依赖读者的推断能力，可以用"注意"、"尽量"、"最好"这类词。约束不行——约束要么是硬的，要么没有意义。

把 CLAUDE.md 想象成单元测试：每一条都在断言一个具体的、可验证的行为。通过的测试是隐形的（Claude 默默做对了）；失败的测试会立刻让你知道（Claude 犯了你已经预见到的错误）。

少即是多。50 条精准的约束，远好过 500 行的愿望清单。

参考来源：*Reddit r/ClaudeCode* 社区讨论、*Claude Code* 官方文档、*Anthropic* 提示工程指南

标签： 人工智能 OpenAI 后端

评论 0



[登录 / 注册](#) 即可发布评论！



暂无评论数据

目录

收起 ^

CLAUDE.md 的最佳实践：为什么你的配置文件基本上是废的

- 先理解一个工程事实：指令容量是有上限的
- 最常见的错误：写了一堆无效指令
- 如何写出有效的指令

✖ 无效写法

✔ 有效写法

三层结构：大多数人完全不知道

- 全局层 ~/.claude/CLAUDE.md
- 项目层 .claude/CLAUDE.md
- 本地层 ./CLAUDE.local.md

实战：精简 CLAUDE.md 的四步法

- Step 1：审计现有内容
- Step 2：从错误倒推规则
- Step 3：控制总长度
- Step 4：定期清理

一个可以直接用的模板

写在最后：写给机器的规则，和写给人的文档，是两件事

精选内容

我 Vibe Coding 了一个 iOS / Flutter 项目的 AI 代码改动检查工具
ZZH_边界之内 · 74阅读 · 2点赞

天天说的 Agent，到底是啥？？？

小锋java1234 · 50阅读 · 1点赞

智能体的构成--深入探讨Anthropic、OpenAI、Perplexity和LangChain究竟在构建什么。

非优秀程序员 · 57阅读 · 1点赞

AI 正在重塑职场：有人乘风破浪，有人悄然掉队

码事漫谈 · 60阅读 · 0点赞

你最喜欢的AI很快就要消失了

安思派Anspire · 367阅读 · 1点赞

为你推荐

告别配置地狱：AGENTS.md如何统一AI编程规则

简瑞_Jerry | 6月前 | 2.7k | 3 | 评论

AI编程 Cursor

OpenClaw 多 Agent 配置实战：踩坑指南与最佳实践

GeraldChen | 2月前 | 8.2k | 11 | 2

人工智能

Claude Code：代理式编码的最佳实践

6confirm | 1年前 | 2.2k | 5 | 评论

前端 后端

CLAUDE.md 最佳实践

Ehtan_Zheng | 15天前 | 267 | 点赞 | 评论

AI编程

详解Claude Code的"大脑"：CLAUDE.md让AI记住你的项目

蚝油菜花 | 8月前 | 2.5k | 5 | 1

人工智能 AI编程 Claude

Claude Code：Agent 编程的最佳实践

是魔丸啊 | 6月前 | 1.1k | 3 | 评论

AI编程 Claude

Claude Code 最佳实践的 8 条黄金法则

程序猿DD | 3月前 | 369 | 1 | 1

Claude

Claude Code 最佳实践：可验证、可治理、可分层的工程现实

Java编程爱好者 | 1月前 | 506 | 3 | 评论

后端

CLAUDE.md 该怎么写：从零开始打造你的 AI 编程搭档说明书

NikoAI编程 | 1月前 | 1.1k | 1 | 1

AI编程 Claude

CLAUDE.md 完全指南

俞凡 | 1月前 | 👁 364 | 👍 1 | 💬 评论

人工智能

AI Agents 的 Skills 文件详解

前端小小栈 | 3月前 | 👁 1.6k | 👍 点赞 | 💬 评论

人工智能

CLAUDE.md 完整模板：让 AI 一次就懂你的项目规范

米小虾 | 1月前 | 👁 403 | 👍 2 | 💬 评论

人工智能

一天一个开源项目（第1篇）：everything-claude-code - 最全的 Claude Code 配置集合

冬奇Lab | 3月前 | 👁 2.7k | 👍 4 | 💬 评论

ClaudeAI编程

Claude Code Skills 入门：什么是技能，为什么你需要它

ccAiHub | 24天前 | 👁 92 | 👍 1 | 💬 评论

ClaudeAI编程

使用Claude Code最需要去做的一件事：与AI签订一份契约（CLAUDE.md）

程序新视界 | 7天前 | 👁 120 | 👍 点赞 | 💬 评论

Claude

